

UNIVERSIDADE FEDERAL DE SANTA CATARINA
CAMPUS REITOR JOÃO DAVID FERREIRA LIMA
CURSO DE SISTEMAS DE INFORMAÇÃO

Guilherme Focassio dos Santos

ANÁLISE E IMPLEMENTAÇÃO DE UM MOTOR DE SINCRONIZAÇÃO DESCENTRALIZADA PARA APLICAÇÕES EM TEMPO REAL

Florianópolis, Santa Catarina, Brasil
2027

Guilherme Focassio dos Santos

ANÁLISE E IMPLEMENTAÇÃO DE UM MOTOR DE SINCRONIZAÇÃO DESCENTRALIZADA PARA APLICAÇÕES EM TEMPO REAL

Trabalho de Conclusão de Curso submetido ao Curso de Sistemas de Informação da Universidade Federal de Santa Catarina para a obtenção do Grau de Bacharel em Sistemas de Informação.
Orientador(a): Odorico Machado Mendizabal, Dr.

Florianópolis, Santa Catarina, Brasil
2027

RESUMO

As arquiteturas de redes de computadores sempre tiveram um forte envolvimento com a velocidade e escalabilidade das soluções distribuídas desenvolvidas por profissionais para problemas em tempo real. Atualmente, sistemas modernos distribuídos enfrentam limitações de escalabilidade, latência e resiliência decorrentes da dependência em arquiteturas cliente-servidor centralizadas. As limitações crescem quando o assunto são sistemas distribuídos interativos em tempo real, ou seja, com usuários ativamente mudando estados, fazendo trocas e atualizações que devem ser sincronizadas com o servidor central. Como uma alternativa a essas propostas, arquiteturas Peer-to-Peer (P2P) surgiram com resoluções e novos desafios. Elas descentralizam o estado, para a carga não ficar sobrecarregada em um só nodo (servidor), mas introduzem complexidades relacionadas a concorrência e divergência de dados. Para avaliarmos o uso e os possíveis endereçamentos de suas nuances será feito um PoC (Proof of Concept) utilizando mapas para sincronização de dados entre múltiplos usuários em tempo real.

Palavras-chaves: Peer-to-Peer. P2P. Cliente-Servidor. CRDTs. Sincronização de Estados.

ABSTRACT

This is the English abstract.

Keywords: Keyword 1. Keyword 2. Keyword 3. Keyword 4. Keyword 5.

LISTA DE TABELAS

Tabela 1 – Comparativo entre libp2p e Iroh 33
Tabela 2 – Comparativo entre Yjs e Automerge 34

LIST OF LISTINGS

*

LISTA DE ABREVIATURAS E SIGLAS

ABNT	Associação Brasileira de Normas Técnicas
abnTeX	ABsurdas Normas para TeX

LISTA DE SÍMBOLOS

Γ	Letra grega Gama
Λ	Lambda
ζ	Letra grega minúscula zeta
$\in$	Pertence

SUMÁRIO

1 INTRODUÇÃO 10
1.1 OBJETIVOS 10
1.1.1 Objetivos gerais 11
1.1.2 Objetivos específicos 11
1.2 ESCOPO DO TRABALHO 12
1.3 MÉTODO DE PESQUISA 12
1.4 O USO DE INTELIGÊNCIA ARTIFICIAL NESTE TRABALHO . . . 12
1.5 ORGANIZAÇÃO DO TRABALHO 13

2 FUNDAMENTAÇÃO TEÓRICA 14
2.1 EVOLUÇÃO HISTÓRICA DAS ARQUITETURAS DISTRIBUÍDAS E DO MODELO P2P 14
2.1.1 P2P de Primeira Geração: A Arquitetura Híbrida do Napster . . 15
2.1.2 P2P de Segunda Geração: Descentralização Não Estruturada, *Flooding* e Supernós 16
2.1.3 P2P de Terceira Geração: Redes Estruturadas, BitTorrent e DHTs (Kademlia) 17
2.2 SINCRONIZAÇÃO DE ESTADO MUTÁVEL EM TEMPO REAL . . 18
2.3 O DILEMA DA CONCORRÊNCIA E CONSISTÊNCIA EVENTUAL FORTE (SEC) 20
2.4 ESTRUTURAS DE DADOS PARA RESOLUÇÃO DESCENTRALIZADA DE CONFLITOS (CRDTS) 21
2.5 O PARADIGMA *LOCAL-FIRST SOFTWARE* E A SOBERANIA DOS DADOS 23

3 ESTADO DA ARTE E TRABALHOS RELACIONADOS 25
3.1 *BLOCKCHAIN* E WEB3: CONSENSO GLOBAL VERSUS CONVERGÊNCIA SEM CONSENSO 25
3.2 PROTOCOLOS SOCIAIS DESCENTRALIZADOS: AT PROTOCOL E NOSTR 25
3.2.1 Nostr: Notes and Other Stuff Transmitted by Relays 26
3.2.2 AT Protocol: Authenticated Transfer Protocol (Bluesky) 26

4 CONTEXTO DO ESTUDO DE CASO E CONCEITOS 28
4.1 LIBP2P: A MODULAR NETWORKING STACK FOR PEER-TO-PEER APPLICATIONS 28
4.2 IROH 28
4.3 YJS 30

4.4 AUTOMERGE 31
4.5 COMPARATIVO E POSICIONAMENTO DO TRABALHO 33

5 ARQUITETURA E DESENVOLVIMENTO DO MOTOR DE SINCRONIZAÇÃO 35

6 AVALIAÇÃO EXPERIMENTAL E RESULTADOS 36

7 CONCLUSÃO 37

REFERÊNCIAS 38

1 INTRODUÇÃO

As redes Peer-to-peer estão na computação como conceito e aplicações desde os primórdios da ARPANET. Na verdade, a própria ARPANET poderia ser descrita como uma rede de comunicação entre nodos - ou “pares” - descentralizada (Crocker, 1969), o que se refletiu na própria criação da internet. A internet como conhecemos hoje é uma comunicação entre múltiplas máquinas, múltiplos pares, para envio e recebimento de recursos. Longe de ser uma novidade recente, o conceito é maduro e fundamenta a comunicação entre sistemas desde a década de 1960.

O conceito evoluiu ao longo de três grandes gerações de redes peer-to-peer, sempre adicionando ou melhorando a solução anterior. Na primeira geração houve a primeira grande implementação dessa arquitetura para resolver o problema de transferência de mídias digitais, ainda com servidores centrais direcionando o tráfego com tabelas de índices. A segunda geração verdadeiramente descentralizou a rede, não dependendo mais de nenhum servidor central. Então, a terceira geração de redes eliminou os principais problemas de flooding, introduziu o upload em chunks e muitos outros problemas de suas antecessoras, se tornando um sucesso global.

Entretanto, essa evolução se restringiu ao envio de arquivos estáticos. Arquivos nunca mudam entre um envio e outro, uma música sempre será o mesmo file, terá o mesmo hash, os mesmos metadados. Para as aplicações do mundo moderno, pautadas no envio e recebimento massivo de dados de aplicações em tempo real, as redes peer-to-peer enfrentam um novo grande desafio: a sincronização de dados em tempo real.

Como uma alternativa para superar essas limitações sem recorrer ao modelo cliente-servidor, este trabalho propõe o desenvolvimento e análise de um Motor de Sincronização Descentralizada para Aplicações em Tempo Real. Para fundamentar e resolver o conflito de divergência sem autoridade central, o motor utiliza estruturas de dados específicas para consistência eventual, como Conflict-free Replicated Data Types (CRDTs) (Shapiro et al., 2011b). Para validar e avaliar o desempenho da solução em termos de latência, resiliência e concorrência, é desenvolvida uma Proof of Concept (PoC) focada na sincronização espacial de mapas interativos entre múltiplos usuários simultâneos.

1.1 OBJETIVOS

Esta seção busca formalizar os objetivos do presente trabalho.

1.1.1 Objetivos gerais

Este trabalho tem como objetivo projetar, implementar e avaliar experimentalmente um motor de sincronização de estado descentralizado (Peer-to-Peer), de baixo nível e agnóstico a domínio, visando garantir a Consistência Eventual Forte (SEC) em aplicações de tempo real por meio da integração de Tipos de Dados Replicados Livres de Conflitos (CRDTs) e ordenação lógica de eventos.

1.1.2 Objetivos específicos

- Estudar os fundamentos teóricos de consistência em sistemas distribuídos descentralizados, analisando os conceitos de Consistência Eventual Forte (SEC), Tipos de Dados Replicados Livres de Conflitos (CRDTs), relógios lógicos e protocolos de fofoca (gossip);
- Analisar os requisitos técnicos e desafios práticos para o desenvolvimento de sistemas de alta performance em tempo real, mapeando os requisitos de paralelismo, controle de concorrência, latência e segurança no ciclo de vida da memória;
- Projetar a arquitetura modular de baixo nível do motor de sincronização, definindo os fluxos de entrada e saída de rede assíncrona, enquadramento (framing) de dados, controle de concorrência multi-thread e o modelo de persistência volátil;
- Implementar o middleware (motor de sincronização) focando na performance de execução, empregando mecanismos eficientes de serialização de dados (zero-copy) e estruturas de dados otimizadas para manipulação concorrente;
- Desenvolver um protótipo funcional de tempo real (aplicação de prova de conceito) que consuma as interfaces expostas pelo motor, demonstrando na prática o comportamento dinâmico e a natureza agnóstica do sistema;
- Avaliar a solução proposta por meio de testes experimentais, analisando métricas de tempo de convergência de réplicas, taxa de transferência de rede (payload de fofoca), latência induzida e consumo de recursos de processamento e memória;
- Documentar as limitações de design identificadas, os aprendizados de engenharia de software aplicados a sistemas distribuídos e as futuras direções para evolução da infraestrutura proposta.

1.2 ESCOPO DO TRABALHO

1.3 MÉTODO DE PESQUISA

Este trabalho utiliza uma pesquisa aplicada, com foco em investigar um problema prático da engenharia de software distribuída: a viabilidade e os desafios de manter estados consistentes em redes de tempo real descentralizadas sem coordenação síncrona global. A abordagem adotada é experimental, baseada no projeto, desenvolvimento e avaliação empírica de um protótipo funcional.

Inicialmente, será realizado um estudo teórico aprofundado a respeito das bases matemáticas de CRDTs (com foco em semilattices superiores e mutações baseadas em delta), relógios de Lamport, relógios vetoriais e algoritmos de disseminação epidêmica (gossip protocols). Além disso, serão revisadas as bibliotecas e técnicas modernas de programação de sockets de rede assíncronos e de alta concorrência na linguagem de programação que será adotada.

Em seguida, será modelada e documentada a arquitetura lógica do motor de sincronização. Esse projeto envolverá o mapeamento das dependências entre os módulos de rede (camada de transporte), de persistência do estado da réplica, de ordenação lógica de eventos e de difusão de atualizações.

Após a etapa de projeto arquitetural, será desenvolvido o motor de sincronização (middleware) sob uma estrutura multi-thread otimizada, implementando as classes e interfaces correspondentes aos CRDTs e relógios lógicos de forma genérica. Concomitantemente, será desenvolvida uma aplicação de prova de conceito (como um simulador distribuído simples ou um quadro colaborativo interativo) cuja única responsabilidade será manipular o estado através do middleware, validando sua natureza agnóstica a domínio.

Por fim, serão conduzidos testes práticos e ensaios experimentais sob diferentes cenários de carga de trabalho e concorrência induzida. Esses ensaios buscarão caracterizar quantitativamente a performance da solução proposta, avaliando as métricas de tempo total de convergência do estado de todas as réplicas ativas, taxa de pacotes descartados por falta de causalidade, consumo de banda de rede (bits/segundo) e consumo de processador e memória RAM nos nós da rede simulada. Os resultados obtidos guiarão a redação das conclusões e limitações técnicas do sistema.

1.4 O USO DE INTELIGÊNCIA ARTIFICIAL NESTE TRABALHO

Em conformidade com as boas práticas de transparência acadêmica, esclarece-se nesta seção como ferramentas de Inteligência Artificial (IA) foram empregadas na elaboração deste trabalho.

Um assistente de IA (Claude, da Anthropic, operando através da ferramenta Claude Code) foi utilizado como apoio em tarefas de natureza instrumental e editorial,

e não na concepção científica da pesquisa. Especificamente, a ferramenta auxiliou:

- Na migração do texto deste documento, originalmente redigido pelo autor em processador de texto, para o template LaTeX da UFSC em conformidade com as normas ABNT, preservando o conteúdo e a redação originais do autor;
- Na configuração do ambiente de desenvolvimento LaTeX local (distribuição TeX Live, editor, extensões) e do fluxo de versionamento do projeto via `git`;
- Na formatação das referências bibliográficas citadas neste capítulo no formato BibTeX e na inserção dos respectivos comandos de citação no texto;
- Na organização geral do repositório do projeto, incluindo a remoção de conteúdo de demonstração herdado do template original.

A definição do tema de pesquisa, a revisão da literatura, a redação do conteúdo técnico e científico, as decisões metodológicas e as conclusões apresentadas ao longo deste trabalho são de autoria e responsabilidade do autor. O uso da IA restringiu-se, portanto, a um papel de ferramenta de produtividade na produção do documento, análogo ao uso de um editor de texto ou de um verificador ortográfico, sem substituir o julgamento científico ou a autoria intelectual da pesquisa.

1.5 ORGANIZAÇÃO DO TRABALHO

Além desta introdução, o restante deste trabalho está organizado da seguinte forma: o capítulo de Desenvolvimento apresenta a fundamentação teórica sobre consistência em sistemas distribuídos descentralizados – CRDTs, relógios lógicos e protocolos de disseminação epidêmica – seguida do projeto da arquitetura modular do motor de sincronização e dos detalhes de sua implementação, com foco em desempenho e concorrência. Em seguida, é apresentado o protótipo de prova de conceito desenvolvido sobre o motor, e a avaliação experimental conduzida sobre ele, caracterizando métricas de convergência, latência, tráfego de rede e consumo de recursos sob diferentes cenários de carga. Por fim, o capítulo de Conclusão retoma os objetivos definidos neste capítulo, discute os resultados obtidos e as limitações identificadas, e aponta direções para trabalhos futuros.

2 FUNDAMENTAÇÃO TEÓRICA

2.1 EVOLUÇÃO HISTÓRICA DAS ARQUITETURAS DISTRIBUÍDAS E DO MODELO P2P

As redes *Peer-to-peer* estão na computação como conceito e aplicações desde os primórdios da ARPANET. Na verdade, a própria ARPANET poderia ser descrita como uma rede de comunicação entre nodos – ou "pares" – descentralizada (Crocker, 1969), o que se refletiu na própria criação da internet. A internet como é conhecida hoje é uma comunicação entre múltiplas máquinas, múltiplos pares, para envio e recebimento de recursos. Longe de ser uma novidade recente, o conceito é maduro e fundamenta a comunicação entre sistemas desde a década de 1960.

Nos anos 90 ocorreu uma transição do que se conhecia anteriormente como ARPANET para a World Wide Web, trazendo com a mudança milhões de usuários leigos buscando e consumindo informações de uma maneira nunca antes vista na história da humanidade. Nesse aspecto, um modelo vigorou, dominou e se consolidou como o padrão desse novo ambiente: o modelo cliente-servidor (Fielding; Taylor, 2000; Tanenbaum; Van Steen, 2007). Para entender os principais motivos do sucesso desse novo modelo, é preciso repensar algumas características das redes *peer-to-peer* que precisariam ser adaptadas para essa nova realidade.

Redes *peer-to-peer* trocam mensagens para definir qual a versão "correta" – muitas vezes subjetiva – de um pacote, dado ou informação. Com o advento da internet, milhões de novas máquinas começaram a processar, enviar e receber esses pacotes/informações a todo momento. Isso, na época, foi um grande problema relacionado à descoberta e busca de recursos entre os nodos, saber onde o recurso ou arquivo estava para fazer o upload ou download (Ripeanu, 2001).

Naquela época, muitas residências tinham altas – para a época – taxas de download, mas baixíssimas taxas de upload. Fazer todas essas máquinas, algumas vezes rápidas, outras lentas, responderem requisições teria uma latência e velocidade muito inferior a simplesmente um *data center* com links dedicados de alta velocidade, capazes de responder milhares de clientes passivos (Saroiu; Gummadi; Gribble, 2002).

Somado a isso, os computadores pessoais não precisavam de IP público estático, portas abertas e nem processar continuamente, garantindo a simplicidade do modelo no lado do cliente (Saltzer; Reed; Clark, 1984).

A adoção do modelo cliente-servidor viabilizou a difusão rápida da Web em seus primeiros anos. Entretanto, no final da década de 1990, uma nova tendência surgiu: a explosão de mídias digitais, impulsionada pela popularização dos arquivos de áudio em MP3. A procura e compartilhamento de músicas se espalhou e trouxe consigo um aumento no número de bytes trafegados. A rede não estava mais apenas baixando e fazendo upload de HTML e arquivos de textos de poucos kbytes, os arquivos de músi-

cas elevaram esse patamar para os megabytes, causando a explosão das contas das infraestruturas dos servidores centrais (Saroiu; Gummadi; Gribble, 2002). Para ilustrar esse gargalo de largura de banda (*egress bandwidth*), em um cenário hipotético no qual um milhão de usuários querem baixar um HTML de 20kbytes o servidor precisará lidar com cerca de 20 Gigabytes de dados de saída, com um arquivo MP3 de 5MB no mesmo servidor, seria necessária uma demanda massiva de 5 Terabytes, um volume desproporcional para a infraestrutura da época.

O volume de dados massivo forçava os servidores centrais a um limite o qual eles não eram capazes de suportar sem encarecer demais o serviço. A centralização do estado e dos recursos fez com que a disponibilidade de todo o sistema ficasse refém do servidor central, um único ponto de falha (*Single Point of Failure*, ou SPOF) (Tanenbaum; Van Steen, 2007). Isso gerou *flash crowds* (picos de tráfego) que derrubaram servidores inteiros por exaustão de CPU, indisponibilizando o serviço para todos os usuários (Padmanabhan; Sripanidkulchai, 2002).

Os pontos negativos do modelo cliente-servidor vieram à tona, escalabilidade, disponibilidade, custo e manutenção eram apenas alguns deles. A crise criou o cenário perfeito para o ressurgimento das redes *Peer-to-peer*, que reemergiam como uma alternativa viável e escalável para o problema, em 1999 surge o Napster.

2.1.1 P2P de Primeira Geração: A Arquitetura Híbrida do Napster

Criado em 1999 por Shawn Fanning e Sean Parker, o Napster como solução de software provou a viabilidade de uma implementação P2P em larga escala (Saroiu; Gummadi; Gribble, 2002). Sua implementação foi realizada para resolver um problema comum da época: a dificuldade de baixar arquivos mp3, como músicas, de sites estáticos ou canais de IRC da época.

O Napster resolveu o gargalo financeiro e o de largura de banda do modelo cliente-servidor na distribuição desses arquivos, na época, pesados. Por exemplo, para uma empresa distribuir um vídeo de 1 Terabyte de dados, seria necessário 1 Terabyte de *egress bandwidth* dos *data centers*. Com o Napster essa infraestrutura de rede, hardware e conexão é repassada para os próprios usuários da rede, os nodos ou *peers*. Os usuários que querem baixar um determinado arquivo fazem essa busca pela rede até encontrar alguém o servindo. A partir disso, cria-se uma conexão entre os dois, onde os recursos computacionais de ambos são usados para o compartilhamento, reduzindo, e muito, o estresse das máquinas centrais dos *data centers* (Adar; Huberman, 2000).

Mesmo com tudo isso, o Napster ainda não era uma rede totalmente descentralizada. Em vez disso, atuava como uma arquitetura híbrida, ou P2P de primeira geração, combinando elementos de centralização com transferência direta de dados (Schollmeier, 2001). Na execução do programa, o cliente se conecta com servidores

centrais mantidos pelo Napster. O cliente, então, envia a lista de todos os MP3 armazenados para compartilhamento, endereço IP e porta de conexão. Os servidores centrais do Napster mantinham em memória índices globais e dinâmicos em tempo real, mapeando milhões de arquivos com respectivos endereços de IP. Quando um usuário procurava por uma música, pelo nome, por exemplo, essa pesquisa era convertida em uma *query* e enviada para o servidor central. O servidor pesquisava em suas tabelas pelo índice e retornava uma lista contando IPs de usuários que estavam provendo aquele conteúdo e suas velocidades de conexão (upload e download). Assim que o usuário escolhia a fonte, a conexão entre ele e o servidor central acabava e começava a conexão TCP direta entre os nodos/*peers*. O arquivo requerido pelo usuário consumidor era, então, enviado do usuário provedor para o seu computador. Embora o problema do gargalo tenha sido resolvido, dois pontos fracos principais finalizaram as operações do Napster em 2001:

- Caso a infraestrutura dos servidores centrais, detentores das tabelas de índices, fosse derrubada, toda a rede como um todo parava de funcionar. Algo análogo a um motorista em uma cidade desconhecida com GPS desligado. Os servidores centrais eram o novo ponto de falha única (SPOF – *Single point of failure*) do sistema (Tanenbaum; Van Steen, 2007);
- Esse ponto único de falha foi justamente o foco principal do ataque contra o Napster realizado por cortes e juízes americanos, que definiram a derrubada desses servidores. Com os servidores fora do ar, a rede perdia o seu propósito.

2.1.2 P2P de Segunda Geração: Descentralização Não Estruturada, *Flooding* e Supernós

Após a queda do Napster, uma rede P2P de primeira geração (híbridas com servidor central), como ficou conhecida posteriormente, surgiram protocolos 100% descentralizados e não estruturados, como o Gnutella (Ripeanu, 2001). Redes P2P de segunda geração são marcadas, principalmente, pela eliminação do servidor central usando *flooding* de mensagens, não existia servidor central nem tabela de índice. Quando um *peer* quer um arquivo, ele envia mensagens diretamente para seus nodos vizinhos, que fazem o mesmo, consecutivamente, até o nodo detentor do arquivo ser encontrado. Uma vez encontrado, uma conexão direta é estabelecida entre ambos para upload ou download do arquivo, a depender da ponta. O claro gargalo é o *flooding*, ou inundação, de mensagens na rede, gerando um tráfego exponencial $O(d^k)$, onde d é o número de conexões e k o TTL. Em uma rede com milhões de nós a internet dos usuários poderia ser consumida apenas com tráfego de terceiros.

Almejando o controle do *flooding*, foram introduzidos o conceito de super nós, uma arquitetura semi-híbrida (Saroiu; Gummadi; Gribble, 2002; Yang; Garcia-Molina,

2003; Liang; Kumar; Ross, 2006). Esses super nós faziam a busca (o *flooding*) entre eles, que possuíam muitos IPs dinâmicos usados para direcionar o fluxo otimizado da mensagem. Como exemplo, pode-se imaginar uma carta enviada pelos correios. Em vez de cada vizinho perguntar para os do lado se aquela é a casa correta, apenas as primeiras casas de cada rua se comunicam, e elas sabem quais casas têm em suas ruas. No momento em que uma dessas primeiras casas avisa que conhece a casa alvo, a primeira casa conta diretamente o endereço dela para o envio da carta, criando-se um caminho muito mais direto até achar a casa correta.

2.1.3 P2P de Terceira Geração: Redes Estruturadas, BitTorrent e DHTs (Kademlia)

As redes P2P de terceira geração, criadas por Bram Cohen em 2001, foram popularizadas por clientes de soluções como utorrent (Cohen, 2003). O bittorrent revolucionou a transferência de dados ao abandonar o modelo vigente, o qual enviava arquivos inteiros de um nodo a outro. A fragmentação em blocos, ou *pieces*, é definida pela divisão em pequenos blocos, um usuário não precisa esperar o download terminar para começar a enviar dados. Enquanto baixa o bloco B de usuário 1, envia o bloco A que já detém para o usuário 2.

A verdadeira descentralização do bittorrent veio à tona somente com o surgimento das DHTs, ou *distributed hash tables*, em especial o protocolo Kademlia (Maymounkov; Mazières, 2002), que permitiu a operação sem rastreamento do bittorrent. Em vez de depender de super nós ou inundações cegas, o Kademlia organiza a rede atribuindo um identificador numérico único (*hash* de 160 bits) tanto para os nós quanto para os arquivos. A localização de um dado é feita calculando uma "distância lógica" matemática entre esses identificadores por meio da operação XOR. Para visualizar o funcionamento de uma DHT, pode-se imaginar um grande catálogo telefônico global dividido em seções. Um usuário não precisa guardar o contato de milhões de pessoas na internet; ele guarda apenas uma lista telefônica reduzida, contendo contatos próximos do seu próprio número e alguns contatos estratégicos para regiões mais distantes da rede. Quando deseja buscar um arquivo específico, o nó envia a requisição para o contato de sua tabela que possui a numeração mais próxima da chave do arquivo desejado. Esse contato, por sua vez, redireciona o pedido para alguém ainda mais próximo, afunilando a busca rapidamente até alcançar o detentor do dado. Esse roteamento estruturado garante que a busca por um arquivo em uma rede com milhões de nós seja concluída em apenas $O(\log N)$ saltos de mensagens, eliminando completamente a tempestade de tráfego (*flooding*) característica das redes de segunda geração.

2.2 SINCRONIZAÇÃO DE ESTADO MUTÁVEL EM TEMPO REAL

As *distributed hash tables* na terceira geração das redes P2P foram um incontestável sucesso. A distribuição massiva de dados descentralizados em blocos entre múltiplos milhões de usuários foi alcançada depois de inúmeros avanços e superações de problemas relacionados a hardware, SO e redes de computadores (Maymounkov; Mazières, 2002; Cohen, 2003). Entretanto, essas grandes redes foram projetadas sob a premissa de uma imutabilidade de dados, um arquivo MP3, ou um vídeo, não mudam de tempos em tempos, seu *hash* sempre será aquele, a depender do algoritmo utilizado. Um outro *hash* representa um arquivo completamente diferente, sempre.

As redes de terceira geração, embora muito bem-sucedidas e evoluídas para a época, não foram projetadas para lidar com dados tão mutáveis. Elas utilizavam um modelo mental chamado de *Single-Writer / Multi-Reader* (SWMR), ou seja, um único escritor e múltiplos leitores. Um arquivo MP3, por exemplo, é escrito pelo usuário que o possui, e todos os outros usuários apenas leem esse arquivo, a rede P2P precisa resolver apenas o problema da distribuição eficiente de leitura (*Multi-Reader*). No início dos anos 2000, grandes empresas, como a Google, desenvolveram aplicações web colaborativas, os quais não poderiam ser regidos pelo modelo SWMR, pois múltiplos usuários poderiam alterar o mesmo dado ao mesmo tempo. Esses editores operavam sob o modelo *Multi-Writer / Multi-Reader* (MWMR).

A Web 2.0 precisou, então, resolver o problema da edição colaborativa em tempo real no fim dos anos 2000, impulsionada pelo Google Docs e Etherpad. A indústria respondeu adotando o paradigma *Operational Transformation*, OT (Ellis; Gibbs, 1989). Em vez de enviar um dado por inteiro, o OT envia apenas as operações realizadas sobre o dado. Quando elas ocorrem concorrentemente, o OT passa as operações por uma função matemática de transformação $T(O_1, O_2)$, que ajusta as posições e índices da segunda operação para que ela faça sentido com base no fato de que O_1 já existia (Sun; Ellis, 1998).

O OT foi implementado em arquiteturas centralizadas, pois era onde as grandes empresas detinham suas soluções para ambientes colaborativos (Nichols et al., 1995). Contudo, ele se torna complexo e quebradiço em redes P2P descentralizadas.

- A dependência de ordenação global síncrona, torna extremamente difícil e pesado determinar o estado correto das operações concorrentes, exigindo algoritmos de consenso distribuídos.

- A explosão de combinação de estados em uma P2P com múltiplos nós, exige que cada nó implemente propriedades matemáticas, conhecidas como *Transformation Properties* (Ressel; Nitsche-Ruhland; Gunzenhäuser, 1996). Manter essas propriedades para cenários com três ou mais nós sem uma autoridade central provou ser cientificamente e praticamente inviável, levando a falhas onde o texto

ou estado final divergia permanentemente entre as réplicas (Imine et al., 2003; Oster et al., 2006).

- O alto custo de memória histórica, pois cada nó precisa manter um histórico de todas as operações realizadas, para que possa aplicar as transformações corretas em caso de divergência. Inviabilizando o uso para execução nos computadores pessoais e celulares da época.

O problema de divergência de estados concorrentes descreve a incapacidade de saber qual evento foi realizado primeiro, ou se eles ocorreram simultaneamente. Soma-se a isso a ausência de um relógio global e a incapacidade de se confiar em um relógio local, cujo cristal de quartzo sofre desvios de frequência imprevisíveis em relação a outros computadores (Tanenbaum; Van Steen, 2007). O relógio físico de um sistema operacional, seu *timestamp*, por assim dizer, pode estar adiantado e, em estratégias do tipo *last-write-wins*, fazer com que seu evento sobrescreva um evento posterior, cujo *timestamp* era menor (Kleppmann, 2017). Também há problemas relacionados com a latência imprevisível da rede, impedindo precisão absoluta de milissegundos (Mills, 1991).

Uma possível solução para esse problema é pensar em causalidade, em vez de adivinhar qual evento vem primeiro. A relação *Happened-Before* foi formulada por Leslie Lamport em 1978 (Lamport, 1978), e define que um evento a aconteceu antes de um evento b, denotado $a \rightarrow b$, se:

- Ambos os eventos ocorreram no mesmo processo e a ocorreu antes de b;
- O evento a é o envio de uma mensagem e o evento b é o recebimento da mesma mensagem;
- Existe um evento c tal que $a \rightarrow c$ e $c \rightarrow b$.

Para implementar essa relação, Lamport propôs os relógios lógicos. Cada nó possui um número inteiro local (L) que é sempre incrementado na realização de um evento, garantindo a causalidade localmente (Lamport, 1978). Porém, esse número precisa ser anexado ao envio de mensagens e, ao receber mensagens, o nó precisa atualizar seu relógio com o valor máximo entre o recebido e o interno, acrescido de 1. A limitação dessa solução é que, se $a \rightarrow b$, então $L(a) < L(b)$, mas o inverso não é garantido: torna-se impossível distinguir, apenas pelos relógios, eventos causalmente relacionados de eventos concorrentes entre diferentes nodos (Fidge, 1988; Mattern, 1989).

Para resolver as limitações dos relógios de Lamport, foram propostos os *Vector Clocks* (VC) (Fidge, 1988; Mattern, 1989). Cada nó mantém um vetor de inteiros, com tamanho igual ao número de nós na rede. Cada posição do vetor representa o contador lógico de cada nó. Ao realizar um evento local, o nó incrementa sua própria posição

no vetor. Ao enviar uma mensagem, o nó anexa seu vetor atual. Ao receber uma mensagem, o nó atualiza seu vetor tomando o máximo, posição a posição, entre seu vetor e o vetor recebido e, em seguida, incrementa sua própria posição, garantindo a preservação da causalidade entre eventos concorrentes. Isso garante uma ordem parcial de causalidade entre os eventos de múltiplos nodos, direta ou indiretamente:

- Se $a \rightarrow b$ pelas duas primeiras regras da relação *Happened-Before*, então B é diretamente dependente de A.
- Se $a \rightarrow c$ e $c \rightarrow b$, então B é indiretamente dependente de A.

Para comparar dois eventos, comparam-se seus vetores posição a posição. Se todas as posições de $V(a)$ são menores ou iguais às de $V(b)$ e ao menos uma é estritamente menor, então $V(a) < V(b)$, o que ocorre se, e somente se, $a \rightarrow b$. Caso nem $V(a) \leq V(b)$ nem $V(b) \leq V(a)$, os eventos são concorrentes ($a \parallel b$), situação que os relógios de Lamport não eram capazes de identificar anteriormente (Mattern, 1989).

2.3 O DILEMA DA CONCORRÊNCIA E CONSISTÊNCIA EVENTUAL FORTE (SEC)

Em uma arquitetura P2P descentralizada, órfã de um servidor geral agindo como uma fonte única da verdade (*Single Source of Truth*), a sincronização de dados em tempo real enfrenta o dilema da divergência de dados. Uma dada informação, alterada por dois usuários ao mesmo tempo, precisa ser sincronizada não só por ambos, mas também por toda a rede, que também pode estar alterando outros dados paralelamente. A latência e a ordenação dos fatos podem levar nodos distintos a terem versões diferentes da realidade, o que pode ser contornado com a reintrodução de um servidor central (Tanenbaum; Van Steen, 2007). O problema é o custo com isso atrelado, isto é, a reintrodução, também, de um ponto único de falha (*Single Point of Failure*).

Os relógios lógicos vistos anteriormente permitem identificar a ordem causal e a concorrência entre eventos, mas não resolvem o conflito em si. Para garantir uma consistência forte, os nodos precisariam se coordenar, por meio de consenso ou de uma ordenação total das operações, aguardando a confirmação dos demais antes de aplicar cada alteração. Em redes com latências variáveis, essa espera se torna muito longa, destruindo a disponibilidade do sistema.

O Teorema CAP, proposto como conjectura por Eric Brewer em 2000 (Brewer, 2000) e provado formalmente por Gilbert e Lynch em 2002 (Gilbert; Lynch, 2002), descreve o dilema da consistência em sistemas distribuídos. Ele afirma que é impossível para um sistema distribuído fornecer simultaneamente mais de duas das três garantias: consistência (C), disponibilidade (A) e tolerância a partições (P). Para o caso das redes P2P, nas quais partições de rede são inevitáveis (P), a escolha, durante uma partição, é entre a consistência forte (C) e a disponibilidade (A) (Brewer, 2012). A escolha de

A garante a usabilidade pelo usuário, mas permite que os nodos divirjam temporariamente. Já a escolha de C garante o uso correto da rede, mas torna a experiência lenta ou indisponível. Mesmo na ausência de partições, há uma segunda troca, entre latência (L) e consistência (C), formalizada pelo modelo PACELC (Abadi, 2012), que é justamente o cenário de redes P2P com latências variáveis.

Em aplicações colaborativas em tempo real, costuma-se optar por AP, ou seja, a disponibilidade e tolerância a partições. Para lidar com a consistência, adota-se a consistência eventual (EC), que garante que todos os nodos eventualmente convergirão para o mesmo estado final, desde que não haja novas atualizações (Vogels, 2009). A consistência eventual forte (SEC) vai além: quaisquer réplicas que tenham recebido o mesmo conjunto de atualizações estão imediatamente no mesmo estado, sem necessidade de reversões ou de coordenação entre os nodos (Shapiro et al., 2011b). Para isso, são utilizadas estruturas de dados específicas, como os Tipos de Dados Replicados Livres de Conflitos (CRDTs) (Shapiro et al., 2011b), que permitem a resolução de conflitos de forma descentralizada e sem a necessidade de um servidor central.

2.4 ESTRUTURAS DE DADOS PARA RESOLUÇÃO DESCENTRALIZADA DE CONFLITOS (CRDTS)

Lidar com divergências de estado em sistemas distribuídos sempre foi um dos principais impasses da área. No início dos anos 2000, essa comunidade provou o Teorema CAP (Brewer, 2000; Gilbert; Lynch, 2002) e, com isso, a impossibilidade de se ter consistência forte, disponibilidade e tolerância a partições simultaneamente. O OT era direcionado para sistemas centralizados e quebrava em P2P (Oster et al., 2006), algoritmos de consenso tradicionais como Paxos (Lamport, 1998) e *locks* forçavam os nós a conversarem entre si, prejudicando a disponibilidade e a latência das soluções. A resolução veio, novamente, mudando a estratégia de pensamento.

Em vez de forçar a rede a concordar sobre a ordem de mensagens, a ideia era atribuir na própria estrutura de dados a capacidade e regras de fusão fundamentadas em propriedades algébricas. Garantindo isso, a ordem em que as mensagens chegavam pela rede parava de importar (Shapiro et al., 2011b). As três propriedades fundamentais são a comutatividade, associatividade e a idempotência.

Comutatividade garante que a ordem de chegada e aplicações de alterações não importa, o resultado final será o mesmo. Associatividade garante que o agrupamento de múltiplos pacotes, por latência de rede, por exemplo, não importa, o resultado final também será o mesmo. Para finalizar, idempotência garante que a aplicação de uma alteração que o nó já possui não alterará o estado final.

Essas propriedades não surgem por acaso. Nos CRDTs baseados em estado, os estados possíveis de uma réplica formam um semirreticulado superior (*join-semilattice*), ou seja, um conjunto parcialmente ordenado no qual quaisquer dois estados possuem

um supremo (*least upper bound*), o menor estado que contém ambos. A operação de fusão (*merge*) de dois estados é justamente esse supremo, que, por definição, é comutativo, associativo e idempotente. Além disso, cada alteração local só pode fazer o estado "crescer" nessa ordem, nunca retroceder (Shapiro et al., 2011b).

A ideia, porém, não surgiu do zero. Algoritmos como o WOOT (Oster et al., 2006), o Treedoc (Preguiça et al., 2009) e o Logoot (Weiss; Urso; Molli, 2009) já exploravam estruturas comutativas para edição colaborativa em redes P2P. Com esses padrões matemáticos, Shapiro e sua equipe propuseram, comprovaram e criaram os Tipos de Dados Replicados Livres de Conflitos (CRDTs) (Shapiro et al., 2011b), provando que era possível alcançar a SEC (consistência eventual forte) em sistemas distribuídos descentralizados, sem a necessidade de um servidor central.

Após a proposição de Shapiro, os CRDTs viraram objeto de estudo, assim como as primeiras redes P2P. Shapiro e sua equipe definiram duas abordagens equivalentes, já que uma consegue emular a outra (Shapiro et al., 2011b), e, anos depois, uma terceira foi proposta. A primeira, os *State-based* CRDTs, ou CvRDTs (*Convergent Replicated Data Types*), foi uma abordagem intuitiva, na qual, a cada alteração, o estado inteiro do CRDT era enviado para os demais nodos, que o fundiam com seu estado local por meio do *merge*. Algo muito resiliente, pois, graças à idempotência, a rede podia perder, duplicar ou reordenar mensagens sem prejudicar a convergência. A desvantagem clara é o uso de largura de banda, um *overhead* gigantesco a depender do tamanho do CRDT.

A segunda, os *Operation-based* CRDTs, ou CmRDTs (*Commutative Replicated Data Types*), enviava apenas a operação realizada, reduzindo o *overhead* de largura de banda. Em compensação, apenas as operações concorrentes precisavam comutar entre si. Entretanto, essa abordagem exigia que a rede entregasse todas as operações exatamente uma única vez e em ordem causal (*causal broadcast*), caso contrário, o estado final divergiria permanentemente. Isso aumentou consideravelmente a complexidade na camada de rede do sistema (Shapiro et al., 2011a).

A terceira, os *Delta-State* CRDTs, surgiu anos depois como um meio termo entre as duas abordagens anteriores. Em vez de enviar o estado inteiro (um grande peso para a rede, mas alta resiliência de consistência) ou apenas a operação (um peso baixo para a rede, mas alta complexidade de entrega), o *Delta-State* CRDT envia apenas o *delta*, ou seja, a diferença entre o estado anterior e o estado atual. Os *deltas* também são elementos do semirreticulado, podendo ser agrupados e fundidos entre si antes do envio. Isso reduz o *overhead* de largura de banda, mantendo a resiliência de consistência, pois o *delta* é aplicado no estado local do nó pelo mesmo *merge*. Ainda assim, para preservar a causalidade, é necessário um protocolo de sincronização entre os nós (*anti-entropy*), garantindo que todos eventualmente convergirão para o mesmo estado final (Almeida; Shoker; Baquero, 2018).

Sobre essas abordagens foram construídos diversos tipos de CRDTs (Shapiro

et al., 2011a). Entre os mais comuns estão os contadores, como o G-Counter, que só pode ser incrementado, e o PN-Counter, que permite incrementos e decrementos; os registradores, como o LWW-Register (*Last-Writer-Wins Register*), que mantém o valor da escrita mais recente; os conjuntos, como o OR-Set (*Observed-Remove Set*), que permite adicionar e remover elementos de forma concorrente; e as sequências, como o RGA (Roh et al., 2011), usadas na edição colaborativa de texto. Esses tipos podem ainda ser combinados em estruturas mais complexas, como mapas e documentos JSON, nos quais cada chave guarda outro CRDT (Kleppmann; Beresford, 2017).

2.5 O PARADIGMA *LOCAL-FIRST SOFTWARE* E A SOBERANIA DOS DADOS

O modelo cliente-servidor é o paradigma histórico da internet há décadas. Em sua égide centralizada se consolidou o modelo *Cloud-First*, onde o estado principal da aplicação, os dados e outras informações dos usuários residem em servidores centralizados, controlados por grandes provedores (empresas, bancos, governos, entre outros) que agem como a única fonte de verdade daqueles dados (*Single Source of Truth*). Embora esse paradigma tenha resolvido os problemas de colaboração em tempo real e compartilhamento de dados, ele tornou as aplicações dependentes de conectividade e latência, além de colocar em risco a privacidade dos dados do usuário.

Alternativamente à abordagem centralizada, Kleppmann, Wiggins et al. (2019) propuseram um novo paradigma, guiado por sete ideais (velocidade, uso em múltiplos dispositivos, funcionamento *offline*, colaboração, longevidade, privacidade e controle do usuário). O *Local-First software* define um modelo não apenas com funcionalidades *offline*, mas com uma verdadeira inversão de hierarquia no armazenamento de dados. A cópia primária dos dados deve residir no computador local do usuário; o servidor, quando existe, é reduzido a um papel secundário de arquivamento, roteamento e auxílio às conexões ponto a ponto.

Conciliar a “independência da rede” com a “colaboração contínua” é uma batalha desafiadora em sistemas distribuídos, uma vez que, *offline*, os estados e ramificações divergem. Kleppmann, Wiggins et al. (2019) apontam os CRDTs como a base mais promissora para conciliar esses dois “ideais”, pois garantem a SEC ao mesclar (*merge*) alterações locais de forma determinística quando a rede é restabelecida, ainda que existam desafios em aberto.

O paradigma *Local-First* traz consigo uma discussão antiga sobre soberania de dados. No modelo estabelecido, o usuário tem, supostamente, controle de seus dados, mas precisa se comunicar por uma API para acessá-los, um direito temporário de acesso, em muitos casos. Caso ocorra um bloqueio de conta, o usuário perde o acesso, mas aqueles dados não são necessariamente deletados pelo provedor e, em incontáveis casos, são usados como produto para terceiros (Zuboff, 2019). A célebre frase “Dados são o novo petróleo” por Clive Humby (Palmer, 2006) ilustra a visão das

empresas sobre esse aspecto.

Ao transferir a base de dados em sua completude para o armazenamento local, o sistema confere ao usuário a verdadeira custódia da informação. No contexto *peer-to-peer*, isso é elevado ao grau máximo de soberania: a centralização da infraestrutura é removida e a comunicação se torna direta entre as máquinas que detêm posse local desses dados, unificando a autonomia do armazenamento local com a topologia descentralizada de rede. Implementações em cima das redes P2P, voltadas à soberania de dados, incluem as *blockchains* e a Web3.

3 ESTADO DA ARTE E TRABALHOS RELACIONADOS

3.1 *BLOCKCHAIN* E WEB3: CONSENSO GLOBAL VERSUS CONVERGÊNCIA SEM CONSENSO

Cada evolução das redes P2P até o presente foi marcada por diferentes aplicações novas surgindo em decorrência de um novo problema resolvido, como, por exemplo, o bittorrent após a inovação no envio em chuncks de arquivos. Neste contexto, nas décadas mais recentes, duas implementações se destacam: aplicações financeiras (bitcoin e ethereum) e aplicações em web 3.0.

Satoshi Nakamoto publicou em 2008 o artigo do Bitcoin, a meta era criar um sistema financeiro sem bancos capaz de realizar transações financeiras de modo quase anônimo. O maior desafio técnico dessa rede era o "problema do gasto duplo": se um usuário detém uma moeda e tenta enviá-la para a pessoa A e para a pessoa B ao mesmo tempo, a rede P2P precisa ter um mecanismo infalível para decidir qual transação vale e qual deve ser anulada. Para endereçar essa questão, o Bitcoin e do Ethereum usam a rede P2P para trocar mensagens (gossip), mas o estado real da rede não muda. Eles agrupam as transações em blocos e usam o poder computacional (Proof-of-Work) ou apostas financeiras (Proof-of-Stake) para garantir quem deve escrever no próximo bloco, atingindo o consenso global síncrono. Toda a rede precisa concordar com uma ordem total e estrita dos eventos. Todos os computadores do mundo processam exatamente a mesma fila de informações.

Para o caso da Web 3.0, o modelo anterior é inviável. Uma aplicação colaborativa não pode esperar segundos em uma fila de operações a cada movimento do mouse ou edição de um documento. Aqui entra a teoria os CRDTs, usados para, de forma deterministica, convergir a rede para o mesmo estado, sem a necessidade de consenso. Se o Usuário A move um marcador para a coordenada X, e o Usuário B move o mesmo marcador para a coordenada Y exatamente no mesmo milissegundo, a rede não precisa travar e fazer uma votação global para decidir qual edição prevalece. O caso colaborativo usa a rede P2P de forma otimista, o computador aceita a mudança localmente na hora (Local-First, zero latência) e, em segundo plano, envia a edição para os outros nós da rede.

ESSA SEÇÃO PRECISA DE UMA MELHORA DE COESÃO, IDEIA E ESTRUTURA! WORK IN PROGRESS.

3.2 PROTOCOLOS SOCIAIS DESCENTRALIZADOS: AT PROTOCOL E NOSTR

Outro caso de uso moderno de protocolos descentralizados são os protocolos sociais. Foram pautados em uma contraestratégia ao modelo das redes sociais da web 2.0, onde as grandes empresas de tecnologia centralizadas controlam, vendem

e espionam dados dos usuários a todo momento. De forma contrária, a indústria e a comunidade *open-source* começaram a buscar protocolos onde o usuário é dono do seu grafo social e dos seus dados.

Atualmente, essas redes sociais estão entre as maiores redes distribuídas em uso. A literatura e a documentação mostram como sincronizar milhões de pequenos eventos diários sem a utilização engessada de uma blockchain. Porém, na prática, alguns ainda dependem de servidores grandes, como o Bluesky (Kleppmann; Frazee et al., 2024). O problema que eles resolvem é o dos "jardins murados" (*walled gardens*) da Web 2.0 (como Twitter e Facebook), onde a empresa central é dona do seu grafo social e dos seus dados (Berners-Lee, 2010). Se o usuário for banido ou a empresa falir, ele perde tudo.

3.2.1 Nostr: Notes and Other Stuff Transmitted by Relays

O Nostr é um protocolo minimalista que concentra o poder no cliente local do usuário em detrimento dos grandes servidores centrais, não é necessário um *login*, criação de conta ou exposição de *e-mail*/senha a terceiros. A identidade do usuário é um par de chaves criptográficas (pública e privada), semelhante ao modelo de identidade das redes P2P. Porém, o Nostr não é P2P: os clientes se conectam a servidores (*relays*) que não conversam entre si. Esses servidores se tornam "burros" agindo apenas como espalhador de mensagens criadas e assinadas criptograficamente nos nós "inteligentes" (computador, aplicativo do celular, televisão inteligente) locais dos usuários. Se algum desses servidores bloquear o usuário por quaisquer razões, seus dados não estão perdidos, basta se conectar a outro servidor para espalhar suas mensagens. Como a chave criptográfica pertence ao usuário, seus seguidores o encontrarão na nova rota, desde que saibam em quais *relays* ele publica (Nostr Protocol, 2026b). Os dados e a identidade pertencem puramente ao nó local do usuário (Nostr Protocol, 2026a).

3.2.2 AT Protocol: Authenticated Transfer Protocol (Bluesky)

O Authenticated Transfer Protocol foi desenvolvido pela equipe do Bluesky (projeto iniciado pelo Twitter em 2019 e independente desde 2021) com participação de pesquisadores de sistemas distribuídos, como Martin Kleppmann (Kleppmann; Frazee et al., 2024). Surge como o protocolo por trás da rede social Bluesky, focado na portabilidade de contas e na separação entre quem guarda os dados e quem mostra o *feed* para o usuário. Alternativamente a um banco de dados centralizado de um grande provedor, usuários têm o direito de hospedar o seu próprio PDS (*Personal Data Server*) em um servidor próprio, que precisa ficar sempre *online*, para guardar seu estado pessoal.

Para organizar o repositório de cada usuário, são empregadas *Merkle Search*

Trees (Auvolat; Taïani, 2019). Elas servem de base para CRDTs de estado e permitem que os indexadores da rede comparem o estado de dois servidores rapidamente e troquem apenas os deltas (as diferenças ou novas postagens), sem precisar copiar o banco de dados inteiro. Como o estado dos dados está separado do restante, os usuários têm o direito a escolher qual algoritmo irá organizar o seu *feed* personalizado, resultando em uma forma de soberania algorítmica (Kleppmann; Frazee et al., 2024).

4 CONTEXTO DO ESTUDO DE CASO E CONCEITOS

4.1 LIBP2P: A MODULAR NETWORKING STACK FOR PEER-TO-PEER APPLICATIONS

Historicamente, aplicações *peer-to-peer* (como Napster ou BitTorrent) possuíam camadas de rede monolíticas, com protocolos de transporte e mecanismos de descoberta de pares rigidamente acoplados à lógica de suas aplicações. Isso criava sistemas inflexíveis, incapazes de se comunicar entre nodos usando diferentes meios de transporte ou ambientes (navegador, servidor, smartphone).

A motivação por trás da libp2p foi criar uma *stack* de rede modular agnóstica. A internet é um ambiente fora de controle, repleto de *firewalls*, roteadores NAT e dispositivos operando em camadas e contextos distintos. A libp2p foi imaginada para integrar esses diferentes sistemas, independentemente de seus protocolos, redes, sistemas operacionais e ambientes distintos (Protocol Labs, 2026). Para isso, ela é composta por uma coleção de módulos que podem ser empilhados e usados em conjunto, conforme a necessidade da aplicação.

Diferente do modelo cliente-servidor tradicional, onde os nós são identificados por endereços de IP, na libp2p cada nó é identificado por um PeerID, derivado da sua chave criptográfica pública (Libp2p, 2026f). O endereço de IP é tratado apenas como uma rota temporária (multiaddr) para chegar àquela identidade (Multiformats, 2026).

A biblioteca suporta diversos meios de transporte, como TCP, QUIC (sobre UDP), WebRTC (para *browsers*) e WebSockets (Libp2p, 2026d). Uma vez que dois nós tenham estabelecido uma conexão segura, é permitido que múltiplos subprotocolos paralelos rodem sobre essa mesma conexão (multiplexação), sem interferir uns nos outros (Libp2p, 2026b). A ferramenta também resolve o clássico problema de encontrar nós, através de módulos intercambiáveis (Libp2p, 2026d).

Já para o problema de NAT, a libp2p oferece três módulos: o AutoNAT, que detecta se o nó está atrás de um NAT (Libp2p, 2026e); o Circuit Relay, que usa um outro nó como intermediário da conexão (Libp2p, 2026a); e o DCUtR, que tenta transformar essa conexão intermediada em uma conexão direta, através de *hole punching* (Libp2p, 2026c).

A forte adoção da libp2p validou o modelo modular para toda uma indústria. Ela atua como infraestrutura de transporte em grandes redes como a camada de consenso do Ethereum (Ethereum Foundation, 2026) e o IPFS (Protocol Labs, 2026).

4.2 IROH

Embora a libp2p e o IPFS (para sistemas de arquivos) tenham se estabelecido como padrões na indústria para redes *peer-to-peer* e Web3, essas ferramentas ainda

são consideradas, para algumas aplicações, complexas. Como alternativa, surgiu uma segunda opção, que evoluiu conceitualmente e respondeu a gargalos da libp2p, com foco em velocidade, simplicidade e dispositivos móveis. O acoplamento entre os componentes da libp2p (DHT, *pubsub*, negociação de transporte) torna o sistema difícil de configurar e de entender (N0 Computer, 2026b).

O Iroh nasceu como um projeto de reescrita do IPFS na linguagem Rust para torná-lo mais rápido. Contudo, para atingir as métricas de desempenho e confiabilidade desejadas, os desenvolvedores perceberam que precisariam quebrar a compatibilidade com as implementações existentes do IPFS (N0 Computer, 2023). A motivação do Iroh é fornecer conexões P2P diretas entre dispositivos, identificados por chave pública, com a menor latência possível e sem a complexidade de configuração da camada de rede (N0 Computer, 2026c).

O Iroh foi construído nativamente sobre o protocolo QUIC (Iyengar; Thomson, 2021), a base do HTTP/3 (Bishop, 2022), descartando o TCP (N0 Computer, 2026c). O QUIC oferece estabelecimento rápido de conexão, criptografia nativa (TLS 1.3) e ausência do bloqueio de cabeça de fila (*head-of-line blocking*) entre *streams*, tornando a troca de mensagens mais rápida, mesmo em redes instáveis, como 4G e 5G.

Diferente de protocolos antigos que usam SHA-256, o iroh-blobs, protocolo de transferência de dados do Iroh, utiliza o algoritmo de hash BLAKE3 (N0 Computer, 2026f). Por ser internamente uma árvore de Merkle, ele permite que os dados sejam processados de forma paralelizada e verificados durante a transmissão (*verified streaming*), aproveitando a velocidade máxima do hardware (O'Connor et al., 2020).

Ele resolve problemas com NAT (roteadores bloqueando conexões de fora), um dos principais de aplicações P2P, com servidores de repasse (*relays*), originalmente chamados de DERP (*Designated Encrypted Relay for Packets*) (N0 Computer, 2026d). A conexão começa passando pelo *relay*, por onde os nós trocam informações de rede e tentam se conectar diretamente (*hole punching*). Se conseguirem, o *relay* sai do caminho; se os *firewalls* forem muito rígidos, o tráfego continua passando por ele. O *relay* não consegue ler os dados, pois a conexão é criptografada de ponta a ponta (N0 Computer, 2026i). O usuário não percebe a diferença, pois ela é abstraída pelo Iroh.

Sobre essa camada de conexão, o Iroh oferece protocolos separados, como o iroh-docs, para sincronização de estados (N0 Computer, 2026h). Ele permite que múltiplos nós editem uma estrutura de dados de chave-valor concorrentemente e a rede cuida de sincronizar as atualizações, através de uma técnica chamada *range-based set reconciliation* (N0 Computer, 2026g; Meyer, 2022).

O Iroh já é utilizado em aplicações reais. O Delta Chat, um aplicativo de mensagens descentralizado (Delta Chat, 2026), usa o Iroh em duas funcionalidades: para transferir o *backup* de um dispositivo para outro através de um QR code (Chatmail, 2026b), e para criar canais de comunicação em tempo real entre os aplicativos webxdc que rodam dentro das conversas, através do iroh-gossip (Chatmail, 2026a). Nos dois

casos, os dispositivos se conectam diretamente, sem que os dados passem pelo servidor de mensagens.

Já a Nous Research utiliza o Iroh na Psyche, uma rede para treinamento distribuído de modelos de IA pela internet. Os nós trocam metadados sobre os resultados do treinamento através do iroh-gossip e baixam os resultados em si através do iroh-blobs. Segundo os autores, o Iroh consegue estabelecer conexões diretas em cerca de 90% dos casos, contra 70% da libp2p, e migra a conexão entre redes sem interrompê-la, graças ao QUIC (Nous Research, 2025).

Por fim, os desenvolvedores do Iroh mantêm uma integração de exemplo com o Automerge, onde o Iroh é usado como camada de rede para sincronizar documentos CRDT entre os nós (N0 Computer, 2026e).

4.3 YJS

A prova acadêmica dos CRDTs resolveram os problemas matemáticos para sincronização de estados, mas consumiam muita memória e CPU no processo. A serialização de grandes JSONs travava navegadores mais fracos, a depender do tamanho dos documentos serializados. O Yjs foi criado por Kevin Jahns para preencher a lacuna entre a teoria acadêmica e alta performance, implemetando uma biblioteca capaz de suportar a edição colaborativa em grande escala sem gargalos de memória e CPU.

Yjs implementa o algoritmo YATA, anteriormente desenhado para evitar o embaralhamento de textos quando usuários diferentes editam os mesmos valores. Ele modela o documento colaborativo como uma lista duplamente encadeada, em vez de árvores. O Run-Length Encoding (RLE) usa codifiação binária compacta para condensar milhares de operações em vetor de poucos bytes. O Yjs é focado nos dados e agnóstico em relação à rede, permitindo acoplar os deltas binários em WebRTC, WebSockets ou malhas P2P.

O Yjs não faz suposições sobre a rede usada nas implementações. Paradigmas como cliente-servidor são tão usáveis quanto *peer-to-peer*: desde que as mudanças sejam comunicadas, ocorrerá a sincronização dos estados. Isso ocorre independentemente de a comunicação ser feita por WebSockets, WebRTC (*browsers*) ou outros *providers* (Jahns, 2026c). Ele suporta edição *offline*, o que permite seguir a filosofia Local-First, onde as alterações são aplicadas primeiramente no local e depois comunicadas aos outros nodos. Eles, por sua vez, aplicam essas alterações, que são mescladas automaticamente pelos CRDTs, abstraídos como estruturas de dados convencionais do JavaScript.

Internamente, todos os tipos compartilhados (*shared types*) são listas. Isso significa que um único algoritmo, o YATA (Nicolaescu et al., 2016), resolve os conflitos de todos os tipos. Cada item tem um ID, formado pelo identificador do cliente e por um relógio de Lamport, e guarda os vizinhos do momento em que foi inserido. Com

isso, dois usuários que inserem no mesmo ponto chegam à mesma ordem final. Um item deletado não desaparece, ele vira um *tombstone* no conjunto de deleções (*delete set*). Para que os metadados não cresçam *ad infinitum*, o Yjs junta em um único item os caracteres digitados em sequência pelo mesmo usuário e descarta o conteúdo dos itens deletados. Em um documento real, com 182 mil inserções e 77 mil deleções, o conjunto de deleções ocupou apenas 4,5 KB (Jahns, 2026b).

Usado por diversas aplicações (Jahns, 2026c), o Yjs é utilizado tanto em aplicações cliente-servidor quanto *peer-to-peer*. Entre os usuários mais conhecidos estão o JupyterLab, para notebooks colaborativos, e o JupyterGIS, um editor colaborativo de sistemas de informação geográfica (Jahns, 2026c; GeoJupyter, 2026). O Room.sh, uma ferramenta de reuniões com funcionalidades embutidas de desenho, edição e codificação, faz uso do *framework* para sincronizar seus vários estados entre múltiplos usuários. Outras aplicações diferentes também contam com uso do Yjs, como o Dynaboard, um construtor colaborativo de aplicações web, e o LegendKeeper, uma ferramenta colaborativa para planejamento de campanhas de RPG (Jahns, 2026c).

4.4 AUTOMERGE

A motivação do Automerge era permitir aos programadores a criação de softwares Local-First manipulando estruturas de dados familiares (listas, *strings*, objetos JSON) sem precisar lidar com os problemas complexos de sistemas distribuídos (Automerge contributors, 2026a). Nascido com foco na experiência do desenvolvedor e na preservação de histórico, o Automerge funciona como um “Git para estruturas de dados” (Automerge contributors, 2026a). Foi criado pelo laboratório Ink & Switch, o mesmo que publicou o artigo fundador do Local-First, com participação de Martin Kleppmann, um dos autores desse artigo, e é baseado na pesquisa sobre CRDTs para JSON (Kleppmann; Wiggins et al., 2019; Kleppmann; Beresford, 2017).

Enquanto o Yjs representa internamente todos os tipos como listas, com foco em desempenho, o Automerge modela o documento como uma árvore JSON aninhada (Kleppmann; Beresford, 2017). Na versão 3.0, o Automerge passou a usar o seu formato colunar comprimido também durante a execução, e não apenas no armazenamento, reduzindo o uso de memória em mais de 10 vezes (Automerge contributors, 2026b). Ele salva o histórico completo das mutações compactando colunas de dados semelhantes (Automerge contributors, 2026b). Já a sincronização envia apenas as alterações que o outro nó ainda não possui (Automerge contributors, 2026a).

Assim como no Git, cada alteração feita em um documento é registrada como uma *change*, parecida com um *commit*. Cada *change* é identificada pelo *hash* SHA-256 do seu conteúdo e aponta para as *changes* das quais depende, formando um grafo acíclico dirigido (DAG). As *changes* mais recentes desse grafo, que ainda não possuem sucessoras, são chamadas de *heads* (Automerge contributors, 2026c). Dentro de cada

change, toda operação recebe um ID formado por um contador e pelo identificador da réplica (*actor ID*), ou seja, um relógio de Lamport. Isso impõe uma ordem total entre as operações: vence o maior contador e, em caso de empate, o maior *actor ID* (Automerge contributors, 2026c).

Na maioria dos casos, a mesclagem é direta. Alterações em objetos ou propriedades diferentes são simplesmente combinadas. Em listas e textos, todas as inserções e deleções são preservadas, e inserções concorrentes na mesma posição recebem a mesma ordem em todos os nós (Automerge contributors, 2026d). Já para valores numéricos, o Automerge oferece o tipo `Counter`, que soma incrementos concorrentes: se dois usuários incrementam um valor 3 ao mesmo tempo, o resultado é 5, e não 4 (Automerge contributors, 2026e).

O único caso sem resolução automática é quando dois usuários alteram a mesma propriedade do mesmo objeto. Nesse caso, o Automerge escolhe um dos valores como vencedor, de forma determinística e igual em todos os nós, sem depender do relógio físico das máquinas. Os valores perdedores não são descartados: eles ficam acessíveis pela função `getConflicts()`, permitindo que a aplicação mostre o conflito ao usuário (Automerge contributors, 2026d).

A sincronização entre dois nós é baseada no trabalho de Kleppmann e Howard (2020). Cada nó guarda o que sabe sobre as *heads* do outro e, a cada mensagem trocada, envia apenas as *changes* que o outro ainda não possui, até que os dois tenham o mesmo histórico (Automerge contributors, 2026g). Por cima disso, o Automerge Repo cuida da parte de infraestrutura: ele salva os documentos localmente através de adaptadores de armazenamento, como o IndexedDB no navegador ou o sistema de arquivos, e os distribui para outros nós através de zero ou mais adaptadores de rede, como o WebSocket (Automerge contributors, 2026f).

Os casos de uso podem variar, mas ele se estende para todas as aplicações Local-First ou que requerem um histórico de edições, como no Git. Aplicações Local-First exigem armazenamento local robusto e controle de versão, como gerenciadores de conhecimento, ferramentas de produtividade *offline* e outros tipos de sistemas onde os usuários necessitam viajar entre os históricos passados de alterações.

Os primeiros casos de uso foram os protótipos do próprio Ink & Switch, apresentados no artigo sobre Local-First (Kleppmann; Wiggins et al., 2019): o Trellis, um quadro Kanban inspirado no Trello, que experimentou uma interface de “viagem no tempo” para navegar pelos estados anteriores do documento; o Pixelpusher, um editor colaborativo de *pixel art*; e o PushPin, um mural colaborativo de mídias mistas, parecido com o Miro, considerado pelos autores o mais completo dos três. Além disso, como visto na seção sobre o Iroh, existe uma integração de exemplo que usa o Iroh como camada de rede para sincronizar documentos do Automerge (N0 Computer, 2026e).

4.5 COMPARATIVO E POSICIONAMENTO DO TRABALHO

WORK IN PROGRESS - É PRECISO AVANÇAR MAIS PARA CONTINUAR ESSE CAPITULO

Tabela 1 – Comparativo entre libp2p e Iroh

Critério	**libp2p**	**Iroh**
Arquitetura	Coleção de módulos empilháveis, conforme a necessidade da aplicação	Núcleo de conexão, com protocolos separados por cima (iroh-blobs, iroh-gossip, iroh-docs)
Implementações	Várias (Go, Rust, JavaScript, entre outras), seguindo as mesmas especificações	Rust
Identidade do nó	PeerID, derivado da chave pública	Chave pública Ed25519
Transportes	TCP, QUIC, WebRTC, WebSockets, entre outros	QUIC
Segurança e multiplexação	Módulos próprios sobre a conexão (ou nativos, no QUIC)	Nativos do QUIC
Travessia de NAT	AutoNAT, Circuit Relay e DCUtR (*hole punching*)	Conexão inicia pelo *relay* e migra para direta via *hole punching*
Conexões diretas (segundo a Nous Research)	Cerca de 70%	Cerca de 90%
Descoberta de pares	Kademlia DHT, mDNS e *rendezvous*	DNS (Pkarr) e, opcionalmente, a DHT do BitTorrent; sem DHT Kademlia própria
Difusão de mensagens	GossipSub	iroh-gossip (HyParView e PlumTree)
Casos de uso	Camada de consenso do Ethereum, IPFS, OrbitDB	Delta Chat, Psyche

Fonte: Elaborado pelo autor, com base em Protocol Labs (2026), libp2p (2026d), n0 Computer (2026c), n0 Computer (2026b), n0 Computer (2026a) e Nous Research (2025).

Tabela 2 – Comparativo entre Yjs e Automerge

Critério	**Yjs**	**Automerge**
Base teórica	YATA (Nicolaescu et al., 2016)	CRDT para JSON (Kleppmann; Beresford, 2017)
Modelo de dados	Todos os tipos são listas internamente	Documento JSON aninhado (mapas, listas, texto, contadores)
Histórico	Descarta o conteúdo dos itens deletados	Guarda o histórico completo, como um grafo de *changes*
Foco	Edição colaborativa em tempo real e desempenho	Versionamento e aplicações Local-First
Rede	Agnóstico, via *providers*	Agnóstico, via Automerge Repo
Tempo para aplicar o histórico real (B4)	5.714 ms	14.326 ms
Tamanho final do documento (B4)	159.929 bytes	129.116 bytes

Fonte: Elaborado pelo autor, com base em Jahns (2026c), Jahns (2026b), Automerge contributors (2026a) e Jahns (2026a).

5 ARQUITETURA E DESENVOLVIMENTO DO MOTOR DE SINCRONIZAÇÃO

6 AVALIAÇÃO EXPERIMENTAL E RESULTADOS

7 CONCLUSÃO

REFERÊNCIAS

ABADI, Daniel J. Consistency tradeoffs in modern distributed database system design: CAP is only part of the story. **Computer**, v. 45, n. 2, p. 37–42, 2012. DOI: 10.1109/MC.2012.33. Citado na p. 21.

ADAR, Eytan; HUBERMAN, Bernardo A. Free riding on Gnutella. **First Monday**, v. 5, n. 10, 2000. Citado na p. 15.

ALMEIDA, Paulo Sérgio; SHOKER, Ali; BAQUERO, Carlos. Delta state replicated data types. **Journal of Parallel and Distributed Computing**, v. 111, p. 162–173, 2018. DOI: 10.1016/j.jpdc.2017.08.003. Citado na p. 22.

AUTOMERGE CONTRIBUTORS. **Automerge**. Disponível em: <https://automerge.org>. Acesso em: 23 out. 2026. Citado nas pp. 31, 34.

AUTOMERGE CONTRIBUTORS. **Automerge 3.0**. Disponível em: <https://automerge.org/blog/automerge-3/>. Acesso em: 27 out. 2026. Citado na p. 31.

AUTOMERGE CONTRIBUTORS. **Automerge Binary Format**. Disponível em: <https://automerge.org/automerge-binary-format-spec/>. Acesso em: 27 out. 2026. Citado nas pp. 31, 32.

AUTOMERGE CONTRIBUTORS. **Conflicts – Automerge Documentation**. Disponível em: <https://automerge.org/docs/reference/documents/conflicts/>. Acesso em: 27 out. 2026. Citado na p. 32.

AUTOMERGE CONTRIBUTORS. **Counters – Automerge Documentation**. Disponível em: <https://automerge.org/docs/reference/documents/counters/>. Acesso em: 27 out. 2026. Citado na p. 32.

AUTOMERGE CONTRIBUTORS. **Repositories – Automerge Documentation**. Disponível em: <https://automerge.org/docs/reference/repositories/>. Acesso em: 27 out. 2026. Citado na p. 32.

AUTOMERGE CONTRIBUTORS. **Sync Protocol (rust/automerge/src/sync.rs)**. GitHub. Disponível em: <https://github.com/automerge/automerge/blob/main/rust/automerge/src/sync.rs>. Acesso em: 27 out. 2026. Citado na p. 32.

AUVOLAT, Alex; TAÏANI, François. Merkle Search Trees: Efficient State-Based CRDTs in Open Networks. In: 38TH IEEE International Symposium on Reliable Distributed Systems (SRDS). [S.l.]: IEEE, 2019. p. 221–230. DOI: 10.1109/SRDS.2019.00032. Citado na p. 27.

BERNERS-LEE, Tim. Long live the Web. **Scientific American**, v. 303, n. 6, p. 80–85, 2010. Citado na p. 26.

BISHOP, Mike. **HTTP/3**. [S.l.*:* s.n.], 2022. RFC 9114. DOI: 10.17487/RFC9114. Citado na p. 29.

BREWER, Eric A. CAP twelve years later: how the “rules” have changed. **Computer**, v. 45, n. 2, p. 23–29, 2012. DOI: 10.1109/MC.2012.37. Citado na p. 20.

BREWER, Eric A. Towards robust distributed systems. In: PROCEEDINGS of the Nineteenth Annual ACM Symposium on Principles of Distributed Computing (PODC). [S.l.]: ACM, 2000. p. 7. Keynote. DOI: 10.1145/343477.343502. Citado nas pp. 20, 21.

CHATMAIL. **Peer channels for realtime communication in webxdcs (src/peer_channels.rs)**. GitHub. Disponível em: <https://github.com/chatmail/core/blob/main/src/peer_channels.rs>. Acesso em: 25 out. 2026. Citado na p. 29.

CHATMAIL. **Transfer a backup to an other device (src/imex/transfer.rs)**. GitHub. Disponível em: <https://github.com/chatmail/core/blob/main/src/imex/transfer.rs>. Acesso em: 25 out. 2026. Citado na p. 29.

COHEN, Bram. Incentives build robustness in BitTorrent. In: WORKSHOP on Economics of Peer-to-Peer Systems. [S.l.*:* s.n.], 2003. Citado nas pp. 17, 18.

CROCKER, S. **RFC 1: Host Software**. [S.l.], 1969. Citado na p. 14.

DELTA CHAT. **Delta Chat, decentralized secure messenger**. Disponível em: <https://delta.chat>. Acesso em: 25 out. 2026. Citado na p. 29.

ELLIS, Clarence A.; GIBBS, Simon J. Concurrency control in groupware systems. **ACM SIGMOD Record**, v. 18, n. 2, p. 399–407, 1989. DOI: 10.1145/66926.66963. Citado na p. 18.

ETHEREUM FOUNDATION. **Phase 0 – Networking**. GitHub. Disponível em: <https://github.com/ethereum/consensus-specs/blob/master/specs/phase0/p2p-interface.md>. Acesso em: 25 out. 2026. Citado na p. 28.

FIDGE, Colin J. Timestamps in message-passing systems that preserve the partial ordering. In: PROCEEDINGS of the 11th Australian Computer Science Conference (ACSC’88). [S.l.*:* s.n.], 1988. p. 56–66. Citado na p. 19.

FIELDING, Roy T.; TAYLOR, Richard N. Principled design of the modern Web architecture. **ACM Transactions on Internet Technology (TOIT)**, v. 1, n. 2, p. 115–150, 2000. Citado na p. 14.

GEOJUPYTER. **JupyterGIS**. GitHub. Disponível em: <https://github.com/geojupyter/jupytergis>. Acesso em: 26 out. 2026. Citado na p. 31.

GILBERT, Seth; LYNCH, Nancy. Brewer's conjecture and the feasibility of consistent, available, partition-tolerant web services. **ACM SIGACT News**, v. 33, n. 2, p. 51–59, 2002. DOI: 10.1145/564585.564601. Citado nas pp. 20, 21.

IMINE, Abdessamad et al. Proving correctness of transformation functions in real-time groupware. In: PROCEEDINGS of the Eighth European Conference on Computer Supported Cooperative Work (ECSCW). [S.l.]: Springer, 2003. p. 277–293. DOI: 10.1007/978-94-010-0068-0_15. Citado na p. 19.

IYENGAR, Jana; THOMSON, Martin. **QUIC: A UDP-Based Multiplexed and Secure Transport**. [S.l.*:* s.n.], 2021. RFC 9000. DOI: 10.17487/RFC9000. Citado na p. 29.

JAHNS, Kevin. **CRDT benchmarks**. GitHub. Disponível em: <https://github.com/dmonad/crdt-benchmarks>. Acesso em: 26 out. 2026. Citado na p. 34.

JAHNS, Kevin. **Yjs Internals**. GitHub. Disponível em: <https://github.com/yjs/yjs/blob/main/INTERNALS.md>. Acesso em: 26 out. 2026. Citado nas pp. 31, 34.

JAHNS, Kevin. **Yjs: A CRDT framework with a powerful abstraction of shared data**. GitHub. Disponível em: <https://github.com/yjs/yjs>. Acesso em: 25 out. 2026. Citado nas pp. 30, 31, 34.

KLEPPMANN, Martin. **Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems**. [S.l.]: O'Reilly Media, 2017. Citado na p. 19.

KLEPPMANN, Martin; BERESFORD, Alastair R. A conflict-free replicated JSON datatype. **IEEE Transactions on Parallel and Distributed Systems**, v. 28, n. 10, p. 2733–2746, 2017. DOI: 10.1109/TPDS.2017.2697382. Citado nas pp. 23, 31, 34.

KLEPPMANN, Martin; FRAZEE, Paul et al. Bluesky and the AT Protocol: Usable Decentralized Social Media. In: PROCEEDINGS of the ACM CoNEXT-2024 Workshop on the Decentralization of the Internet (DIN '24). [S.l.]: ACM, 2024. DOI: 10.1145/3694809.3700740. Citado nas pp. 26, 27.

KLEPPMANN, Martin; HOWARD, Heidi. **Byzantine Eventual Consistency and the Fundamental Limits of Peer-to-Peer Databases**. [S.l.*:* s.n.], 2020. arXiv:2012.00472. DOI: 10.48550/arXiv.2012.00472. Citado na p. 32.

KLEPPMANN, Martin; WIGGINS, Adam et al. Local-first software: you own your data, in spite of the cloud. In: PROCEEDINGS of the 2019 ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software (Onward!) [S.l.]: ACM, 2019. p. 154–178. DOI: 10.1145/3359591.3359737. Citado nas pp. 23, 31, 32.

LAMPORT, Leslie. The part-time parliament. **ACM Transactions on Computer Systems**, v. 16, n. 2, p. 133–169, 1998. DOI: 10.1145/279227.279229. Citado na p. 21.

LAMPORT, Leslie. Time, clocks, and the ordering of events in a distributed system. **Communications of the ACM**, v. 21, n. 7, p. 558–565, 1978. Citado na p. 19.

LIANG, Jian; KUMAR, Rakesh; ROSS, Keith W. The FastTrack overlay: a measurement study. **Computer Networks**, v. 50, n. 6, p. 842–858, 2006. Citado na p. 17.

LIBP2P. **Circuit Relay v2**. GitHub. Disponível em: <https://github.com/libp2p/specs/blob/master/relay/circuit-v2.md>. Acesso em: 25 out. 2026. Citado na p. 28.

LIBP2P. **Connection Establishment in libp2p**. GitHub. Disponível em: <https://github.com/libp2p/specs/blob/master/connections/README.md>. Acesso em: 25 out. 2026. Citado na p. 28.

LIBP2P. **Direct Connection Upgrade through Relay**. GitHub. Disponível em: <https://github.com/libp2p/specs/blob/master/relay/DCUtR.md>. Acesso em: 25 out. 2026. Citado na p. 28.

LIBP2P. **libp2p specification**. GitHub. Disponível em: <https://github.com/libp2p/specs>. Acesso em: 25 out. 2026. Citado nas pp. 28, 33.

LIBP2P. **NAT Discovery**. GitHub. Disponível em: <https://github.com/libp2p/specs/blob/master/autonat/README.md>. Acesso em: 25 out. 2026. Citado na p. 28.

LIBP2P. **Peer Ids and Keys**. GitHub. Disponível em: <https://github.com/libp2p/specs/blob/master/peer-ids/peer-ids.md>. Acesso em: 25 out. 2026. Citado na p. 28.

MATTERN, Friedemann. Virtual time and global states of distributed systems. In: COSNARD, Michel et al. (ed.). **Parallel and Distributed Algorithms: Proceedings of the International Workshop on Parallel and Distributed Algorithms**. [S.l.]: North-Holland, 1989. p. 215–226. Citado nas pp. 19, 20.

MAYMOUNKOV, Petar; MAZIÈRES, David. Kademlia: A peer-to-peer information system based on the XOR metric. In: INTERNATIONAL Workshop on Peer-to-Peer Systems. [S.l.]: Springer, Berlin, Heidelberg, 2002. p. 53–65. Citado nas pp. 17, 18.

MEYER, Aljoscha. **Range-Based Set Reconciliation**. [S.l.*:* s.n.], 2022. arXiv:2212.13567. DOI: 10.48550/arXiv.2212.13567. Citado na p. 29.

MILLS, David L. Internet time synchronization: the Network Time Protocol. **IEEE Transactions on Communications**, v. 39, n. 10, p. 1482–1493, 1991. DOI: 10.1109/26.103043. Citado na p. 19.

MULTIFORMATS. **multiaddr**. GitHub. Disponível em: <https://github.com/multiformats/multiaddr>. Acesso em: 25 out. 2026. Citado na p. 28.

N0 COMPUTER. **A new direction for iroh**. 17 fev. 2023. Disponível em: <https://www.iroh.computer/blog/a-new-direction-for-iroh>. Acesso em: 25 out. 2026. Citado na p. 29.

N0 COMPUTER. **Address Lookup – iroh Documentation**. Disponível em: <https://www.iroh.computer/docs/concepts/discovery>. Acesso em: 27 out. 2026. Citado na p. 33.

N0 COMPUTER. **FAQ – iroh Documentation**. Disponível em: <https://www.iroh.computer/docs/faq>. Acesso em: 25 out. 2026. Citado nas pp. 29, 33.

N0 COMPUTER. **iroh**. GitHub. Disponível em: <https://github.com/n0-computer/iroh>. Acesso em: 25 out. 2026. Citado nas pp. 29, 33.

N0 COMPUTER. **iroh CHANGELOG**. GitHub. Disponível em: <https://github.com/n0-computer/iroh/blob/main/CHANGELOG.md>. Acesso em: 25 out. 2026. Citado na p. 29.

N0 COMPUTER. **iroh-automerge**. GitHub. Disponível em: <https://github.com/n0-computer/iroh-examples/tree/main/iroh-automerge>. Acesso em: 25 out. 2026. Citado nas pp. 30, 32.

N0 COMPUTER. **iroh-blobs**. GitHub. Disponível em: <https://github.com/n0-computer/iroh-blobs>. Acesso em: 25 out. 2026. Citado na p. 29.

N0 COMPUTER. **iroh-docs**. GitHub. Disponível em: <https://github.com/n0-computer/iroh-docs>. Acesso em: 25 out. 2026. Citado na p. 29.

N0 COMPUTER. **Protocols – iroh Documentation**. Disponível em: <https://www.iroh.computer/docs/concepts/protocols>. Acesso em: 25 out. 2026. Citado na p. 29.

N0 COMPUTER. **Relays – iroh Documentation**. Disponível em: <https://www.iroh.computer/docs/concepts/relay>. Acesso em: 25 out. 2026. Citado na p. 29.

NICHOLS, David A. et al. High-latency, low-bandwidth windowing in the Jupiter collaboration system. In: PROCEEDINGS of the 8th Annual ACM Symposium on User Interface Software and Technology (UIST). [S.l.]: ACM, 1995. p. 111–120. DOI: 10.1145/215585.215706. Citado na p. 18.

NICOLAESCU, Petru et al. Near Real-Time Peer-to-Peer Shared Editing on Extensible Data Types. In: PROCEEDINGS of the 19th International Conference on Supporting Group Work (GROUP '16). [S.l.]: ACM, 2016. p. 39–49. DOI: 10.1145/2957276.2957310. Citado nas pp. 30, 34.

NOSTR PROTOCOL. **NIP-01: Basic protocol flow description**. GitHub. Disponível em: <https://github.com/nostr-protocol/nips/blob/master/01.md>. Acesso em: 23 out. 2026. Citado na p. 26.

NOSTR PROTOCOL. **NIP-65: Relay List Metadata**. GitHub. Disponível em: <https://github.com/nostr-protocol/nips/blob/master/65.md>. Acesso em: 23 out. 2026. Citado na p. 26.

NOUS RESEARCH. **Democratizing AI: The Psyche Network Architecture**. 14 maio 2025. Disponível em: <https://nousresearch.com/nous-psyche/>. Acesso em: 25 out. 2026. Citado nas pp. 30, 33.

O'CONNOR, Jack et al. **BLAKE3: One function, fast everywhere**. [S.l.: s.n.], 2020. Disponível em: <https://github.com/BLAKE3-team/BLAKE3-specs>. Citado na p. 29.

OSTER, Gérald et al. Data consistency for P2P collaborative editing. In: PROCEEDINGS of the 2006 20th Anniversary Conference on Computer Supported Cooperative Work (CSCW). [S.l.]: ACM, 2006. p. 259–268. DOI: 10.1145/1180875.1180916. Citado nas pp. 19, 21, 22.

PADMANABHAN, Venkata N.; SRIPANIDKULCHAI, Kunwadee. The case for cooperative networking. In: INTERNATIONAL Workshop on Peer-to-Peer Systems (IPTPS). [S.l.]: Springer, Berlin, Heidelberg, 2002. p. 178–190. Citado na p. 15.

PALMER, Michael. **Data is the new oil**. ANA Marketing Maestros. 3 nov. 2006. Disponível em: <https://ana.blogs.com/maestros/2006/11/data_is_the_new.html>. Acesso em: 21 out. 2026. Citado na p. 23.

PREGUIÇA, Nuno et al. A commutative replicated data type for cooperative editing. In: PROCEEDINGS of the 29th IEEE International Conference on Distributed Computing Systems (ICDCS). [S.l.]: IEEE, 2009. p. 395–403. DOI: 10.1109/ICDCS.2009.20. Citado na p. 22.

PROTOCOL LABS. **libp2p: A modular network stack**. Disponível em: <https://libp2p.io>. Acesso em: 25 out. 2026. Citado nas pp. 28, 33.

RESSEL, Matthias; NITSCHE-RUHLAND, Doris; GUNZENHÄUSER, Rul. An integrating, transformation-oriented approach to concurrency control and undo in group editors. In: PROCEEDINGS of the 1996 ACM Conference on Computer Supported Cooperative Work (CSCW). [S.l.]: ACM, 1996. p. 288–297. DOI: 10.1145/240080.240305. Citado na p. 18.

RIPEANU, Matei. Peer-to-peer architecture case study: Gnutella network. In: PROCEEDINGS of the First International IEEE/ACM Workshop on Grid Computing (Grid 2001). [S.l.*:* s.n.], 2001. Citado nas pp. 14, 16.

ROH, Hyun-Gul et al. Replicated abstract data types: building blocks for collaborative applications. **Journal of Parallel and Distributed Computing**, v. 71, n. 3, p. 354–368, 2011. DOI: 10.1016/j.jpdc.2010.12.006. Citado na p. 23.

SALTZER, Jerome H.; REED, David P.; CLARK, David D. End-to-end arguments in system design. **ACM Transactions on Computer Systems (TOCS)**, v. 2, n. 4, p. 277–288, 1984. Citado na p. 14.

SAROIU, Stefan; GUMMADI, P. Krishna; GRIBBLE, Steven D. A measurement study of peer-to-peer file sharing systems. In: PROCEEDINGS of the 2nd ACM SIGCOMM Workshop on Internet Measurement. [S.l.*:* s.n.], 2002. Citado nas pp. 14–16.

SCHOLLMEIER, Rüdiger. A definition of peer-to-peer networking for the classification of peer-to-peer architectures and applications. In: PROCEEDINGS First International Conference on Peer-to-Peer Computing (P2P 2001). [S.l.]: IEEE, 2001. p. 101–102. Citado na p. 15.

SHAPIRO, Marc et al. **A comprehensive study of convergent and commutative replicated data types**. [S.l.], 2011. Disponível em: <https://hal.inria.fr/inria-00555588>. Citado na p. 22.

SHAPIRO, Marc et al. Conflict-free replicated data types. In: SYMPOSIUM on Self-Stabilizing Systems. [S.l.]: Springer, Berlin, Heidelberg, 2011. p. 386–400. Citado nas pp. 10, 21, 22.

SUN, Chengzheng; ELLIS, Clarence. Operational transformation in real-time group editors: issues, algorithms, and achievements. In: PROCEEDINGS of the 1998 ACM

Conference on Computer Supported Cooperative Work (CSCW). [S.l.]: ACM, 1998. p. 59–68. DOI: 10.1145/289444.289469. Citado na p. 18.

TANENBAUM, Andrew S.; VAN STEEN, Maarten. **Distributed Systems: Principles and Paradigms**. [S.l.]: Prentice-Hall, 2007. Citado nas pp. 14–16, 19, 20.

VOGELS, Werner. Eventually consistent. **Communications of the ACM**, v. 52, n. 1, p. 40–44, 2009. DOI: 10.1145/1435417.1435432. Citado na p. 21.

WEISS, Stéphane; URSO, Pascal; MOLLI, Pascal. Logoot: a scalable optimistic replication algorithm for collaborative editing on P2P networks. In: PROCEEDINGS of the 29th IEEE International Conference on Distributed Computing Systems (ICDCS). [S.l.]: IEEE, 2009. p. 404–412. Citado na p. 22.

YANG, Beverly; GARCIA-MOLINA, Hector. Designing a super-peer network. In: PROCEEDINGS of the 19th International Conference on Data Engineering (ICDE). [S.l.]: IEEE, 2003. p. 49–60. Citado na p. 16.

ZUBOFF, Shoshana. **The Age of Surveillance Capitalism: The Fight for a Human Future at the New Frontier of Power**. New York: PublicAffairs, 2019. Citado na p. 23.